

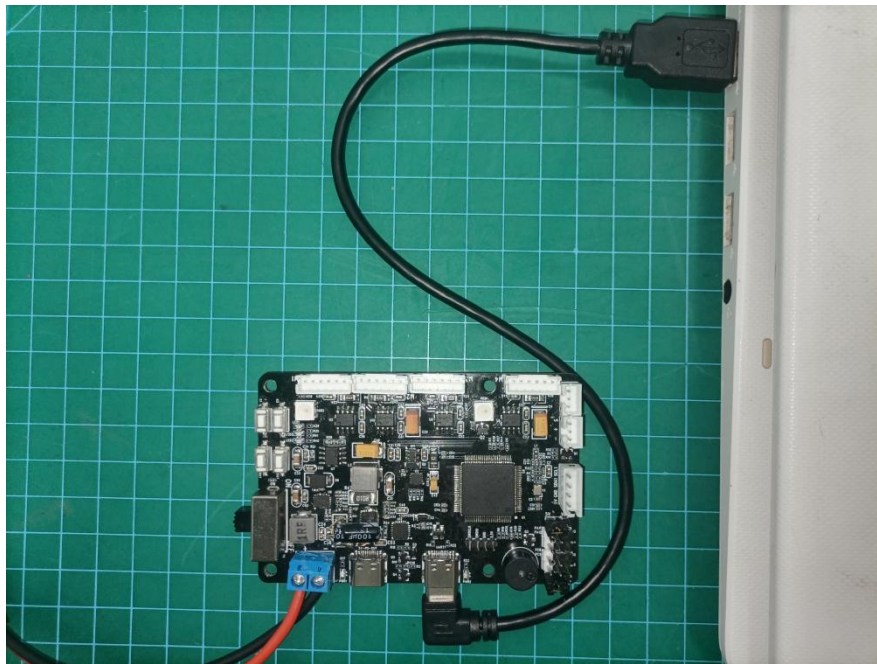
Lesson 5 RRCLite QMI8658 Accelerometer and Gyroscope Source Code Analysis

1. Program Description

This chapter focuses on reading data from the built-in MPU6050 on the STM32 development board, calculating the current attitude of the board, and outputting the data via the serial port 1 to the PC.

2. Hardware Connection

Connect the RRCLite controller to the PC via USB cable through UART 1. The hardware connection diagram is as follows:



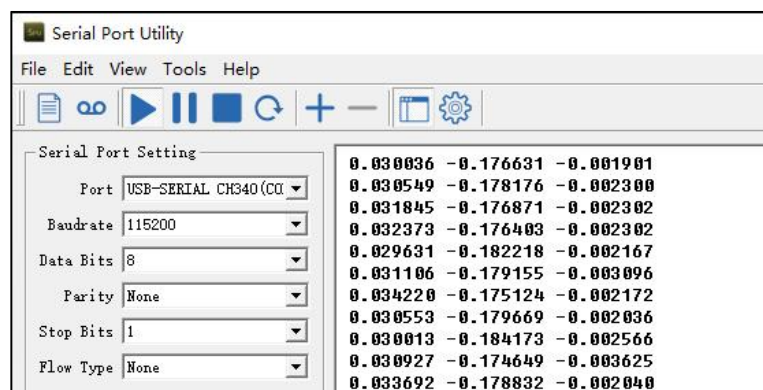
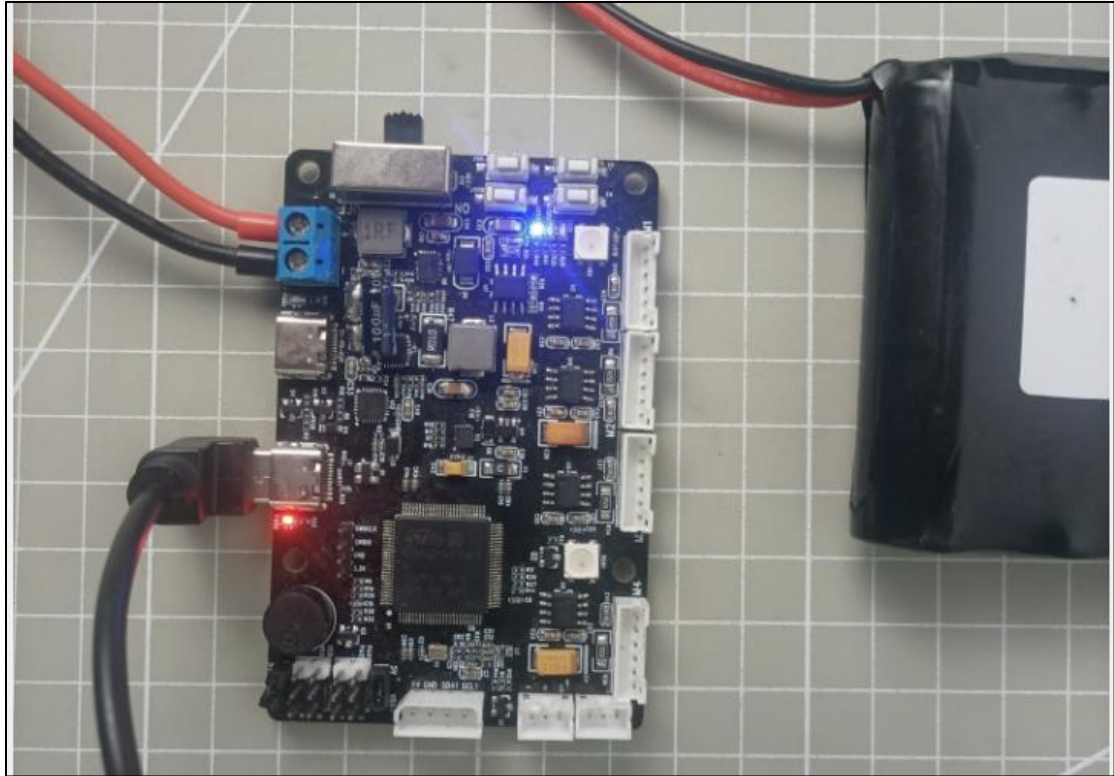
3. Program Download

The example for this section is "RRCLite_IMU".

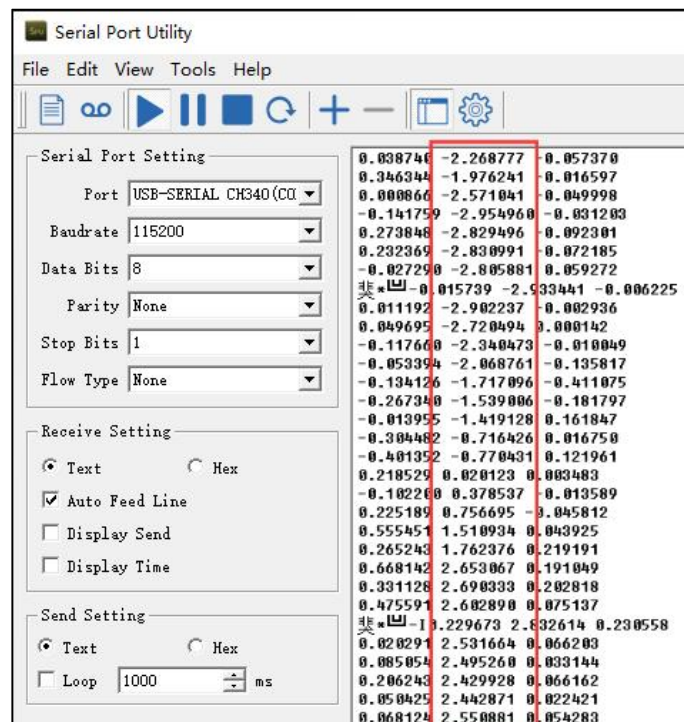
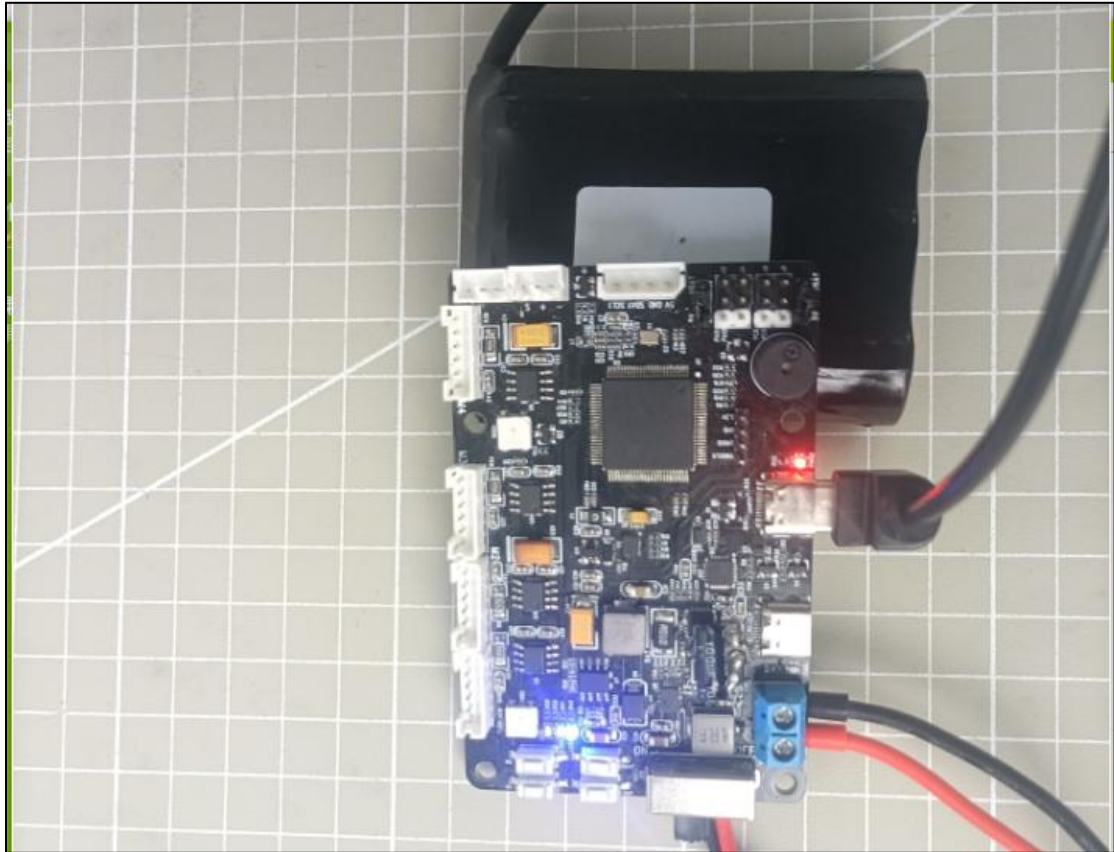
4. Program Outcome

After downloading the program, press the RESET button to see the real-time attitude data of the development board displayed on the OLED screen.

After reset, the data displayed for the three axes is basically zero (with some minor errors), as shown below:



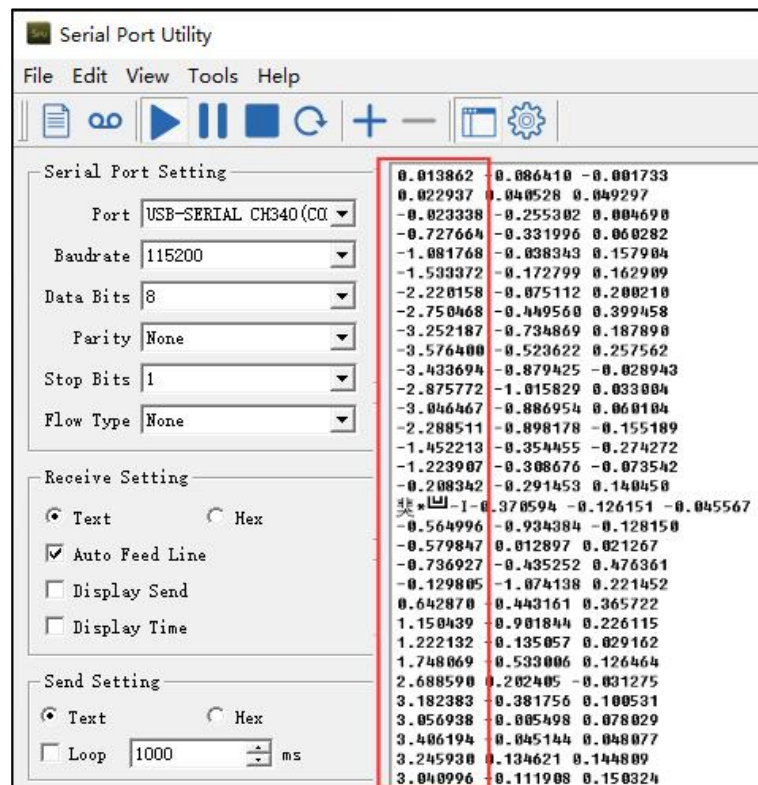
For forward and backward tilting, mainly the Roll data changes.



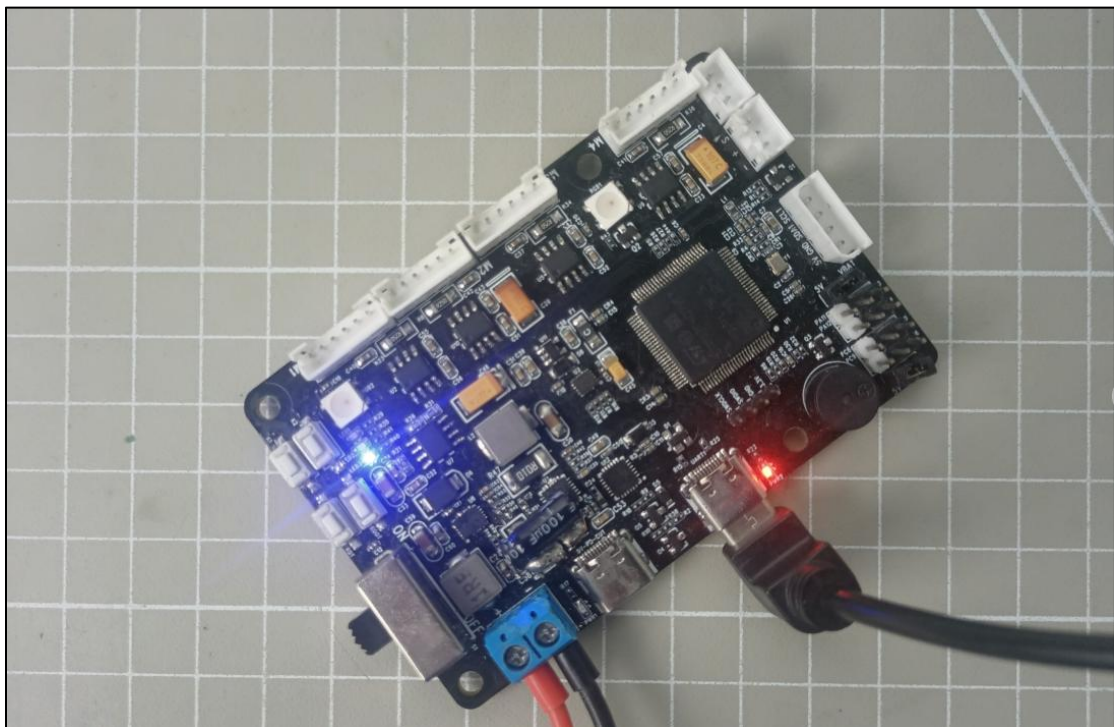
For left and right tilting, mainly the Pitch data changes.

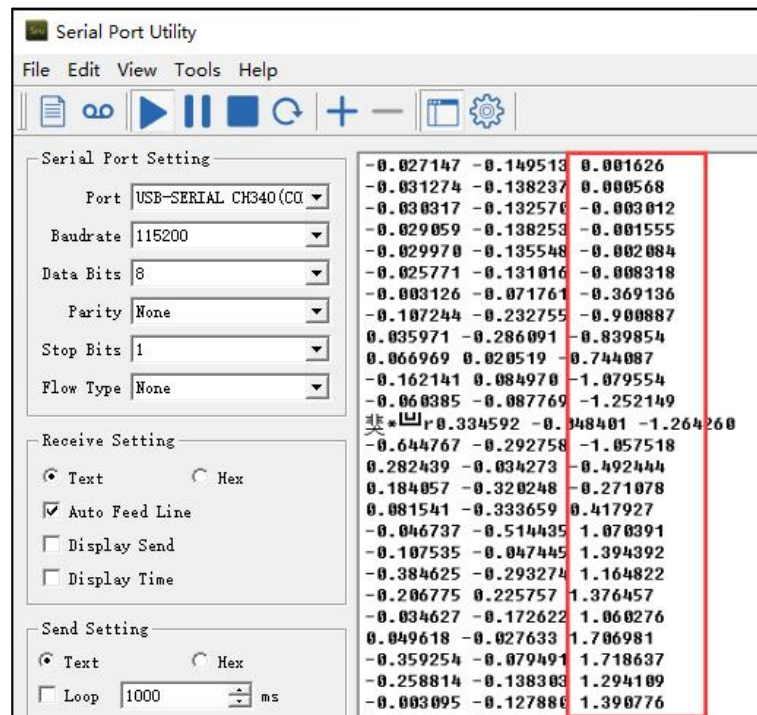
Note: the effect shown below can also be achieved with a 7.4V battery:





For plane rotation, mainly the Yaw data changes.

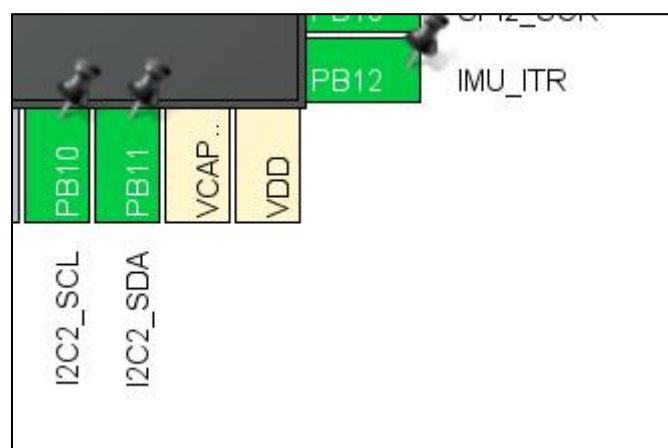




5. Pin Configuration

Use STM32CubeMX software to view the pin information. The microcontroller communicates with the IMU using I2C, and the IMU sends interrupt signals for the microcontroller to collect IMU data.

The corresponding I2C chip pins are PB10 and PB11; the interrupt pin is PB12.

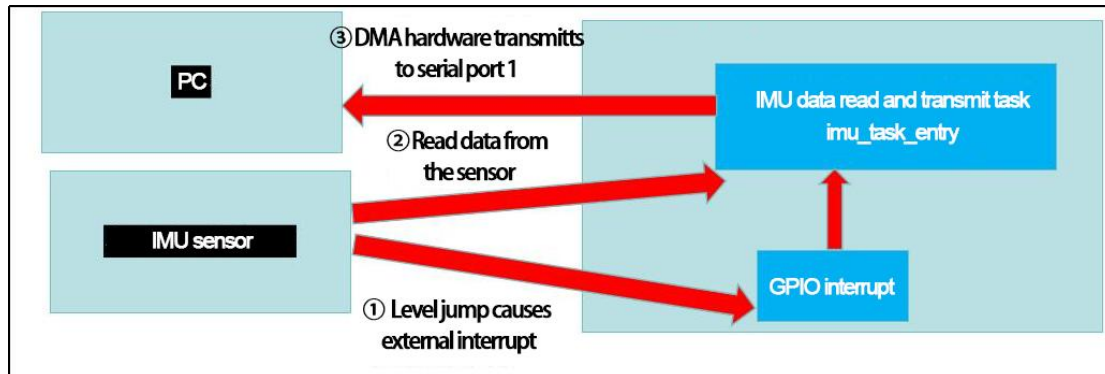


6. Source Code Analysis

The source code for this program is under the same directory as this lesson.

The main program framework is designed as follows:

The interrupt analysis is mainly based on the Core/Src/stm32f4xx_it.c file, and the display task is analyzed based on the Hiwonder/System/app.c file.



The analysis is divided into two parts:

- 1.External interrupt
- 2.The task of reading and forwarding IMU data

6.1 External Interrupt

In the Core/Src/stm32f4xx_it.c file, the EXTI15_10_IRQHandler() function handles the external interrupt. It sends a semaphore signal to unlock the IMU reading block state, as shown below:

```

/**
 * @brief This function handles EXTI line[15:10] interrupts.
 */
void EXTI15_10_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI15_10_IRQn 0 */
    extern osSemaphoreId_t IMU_data_readyHandle;
    if(__HAL_GPIO_EXTI_GET_IT(IMU_ITR_Pin) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(IMU_ITR_Pin);
        osSemaphoreRelease(IMU_data_readyHandle);
    }
    /* USER CODE END EXTI15_10_IRQn 0 */
    /* USER CODE BEGIN EXTI15_10_IRQn 1 */
    /* USER CODE END EXTI15_10_IRQn 1 */
}
    
```

6.2 The task of reading and forwarding IMU data

The reading task initializes the IMU and its parameters, then loops to wait for reading IMU data.

```
void imu_task_entry(void *argument)
{
    extern osSemaphoreId_t IMU_data_readyHandle;
    extern osMutexId_t oled_mutexHandle;

    if(begin() == 0)
    {
        printf("qmi8658_init fail");
    }
    //
    osDelay(100);
    PacketReportIMU_Raw_TypeDef report;
    for(;;) {
        osSemaphoreAcquire(IMU_data_readyHandle, osWaitForever);
        read_xyz(report.array.accel_array, report.array.gyro_array);
        //
        printf("%d,%d,%d\r\n", (int)report.array.accel_array[0], (int)report.array.accel_array[1], (int)report.array.accel_array[2]);
        packet_transmit(&packet_controller, PACKET_FUNC_IMU, &report, sizeof(PacketReportIMU_Raw_TypeDef));
        osDelay(50);
    }
}
#endif
```

The `osSemaphoreAcquire()` function in the RTOS system acquires the semaphore:

Parameter 1: Semaphore handle (here, the IMU semaphore handle)

Parameter 2: Timeout duration (ms), with `osWaitForever` meaning to wait indefinitely.

If there is no semaphore in the queue, the function waits and executes other tasks. When a semaphore is available, it retrieves it and continues running the subsequent program.

The function “`read_xyz()`” is used to read and update data from the IMU. Parameter 1 is the member variable of the IMU accelerometer to be written for synchronization. Parameter 2 is the member variable of the IMU gyroscope to be written for synchronization. After execution, the new IMU posture parameters can be obtained.